

Creating a PHP User Interface for Manipulating MySQL Databases

Marija Mojsilović^{1*}  [0009-0004-6818-2450], Goran Miodragović¹  [0009-0009-4794-8898],
Selver Pepić¹  [0000-0002-7168-1881] and Muzafer Saračević²  [0000-0003-2577-7927]

¹ Academy of Professional Studies Sumadija, College in Trstenik, Trstenik, Serbia

² University of Novi Pazar, Department of Computer Sciences, Novi Pazar, Serbia

* mmojsilovic@asss.edu.rs

Abstract: *In today's digital environment, accessing and manipulating data are crucial aspects of various web applications. This paper explores the process of creating a PHP user interface for efficient manipulation of MySQL databases. Accessing databases through web applications requires careful planning and implementation to ensure security, performance, and practicality. Using PHP as the server-side language and MySQL as the database, we delve into methods for creating an interactive user interface that allows users to read, write, update, and delete data in the database. The focus of the paper is on the application of basic PHP functionalities and techniques such as forms, queries, and error handling to enable a user interface that is intuitive to use and reliable in operation. Through implementation examples and performance evaluation, the key aspects of the process of creating a PHP user interface for manipulating MySQL databases are highlighted, providing guidance for optimization and improving efficiency in developing similar systems. This paper contributes to understanding techniques and practices for creating secure and functional web applications that communicate with databases, with a special focus on PHP and MySQL technologies*

Keywords: *PHP; MySQL; Web application; User interface*

1. INTRODUCTION

In today's digital age, an increasing number of applications require efficient data handling through the user interface to provide users with a rich usage experience. In web development, PHP (Hypertext Preprocessor) is often used as a server-side language for dynamically generating content [1], while MySQL is frequently used as a database for storing and managing data. Connecting PHP with a MySQL database allows developers to create powerful web applications that offer users an intuitive environment for data interaction [1]. Examining the execution speed of computers using the PHP programming language and database enables the optimization of web application performance, making them faster and more efficient in processing and managing data [2].

This paper explores the process of creating a user interface for working with a MySQL database using the PHP programming language. We focus on the analysis of techniques, tools, and practices that enable efficient PHP-MySQL integration for executing queries, displaying data, and managing user interactions [2].

In the first part, we discuss the basics of PHP and MySQL, highlighting their key features and roles in web application development. Then, we explore the principles of user interface design that facilitate the

creation of intuitive and functional user experiences. After that, we provide a detailed analysis of the process of connecting to a MySQL database through PHP, examining various methods and techniques used to establish the connection and execute SQL queries [3].

Additionally, we discuss the security aspects of PHP applications that use a MySQL database, emphasizing the importance of implementing security mechanisms to protect data from potential attacks and information leaks. The use of PHP is not limited to database management; it is also widely used in cryptography for secure encryption and decryption of data, ensuring the protection of confidential information in web applications [4].

The significance of this research lies in the context of digital transformation and the development of web technologies, highlighting the need for further exploration and improvement of practices in creating PHP user interfaces for working with a MySQL database.

2. USER INTERFACE

User Experience (UX) design plays a crucial role in creating an efficient user interface, laying the foundation for interaction between users and the application [3]. The central goal of UX design is to understand the needs, expectations, and

experiences of users in order to create a user experience that is not only functional but also intuitive and enjoyable to use [5]. This involves careful research of the target audience, identifying their goals and challenges, as well as timely collection and analysis of feedback to iteratively improve the user experience. Through effective UX design [5], users experience comfort and increased confidence while using the application, resulting in higher user satisfaction and loyalty.

Aesthetics and visual design are key factors in creating a user interface that attracts users and provides them with a pleasant user experience. Through careful selection of colors, typography, icons, and graphic elements, designers can create a subtle and coherent aesthetic identity that reflects the application's brand and facilitates user orientation [6]. In addition to improving aesthetics, visual design also plays a crucial role in guiding the user's gaze to important interface elements and in emphasizing functionality and interactivity. Through a harmonious combination of form and function, aesthetics and visual design contribute to creating a user interface that inspires, engages, and leaves a lasting impression on users.

Adaptive design is a key element of modern web development that allows the user interface to adapt to different devices and screens. Through the use of media queries, flexible grids, and other techniques, designers can ensure that the application looks and functions seamlessly on desktop computers, tablets, and mobile phones. This approach enables users to have a consistent experience with the application regardless of the device they use, providing them with the freedom to access content and features wherever they are.

Using icons and symbols in the user interface can significantly improve the ease of use and aesthetics of the application [6]. By using clear and consistent icons, designers can facilitate users' understanding of the interface while adding aesthetic value and enhancing the overall impression of the application.

User interface testing is a crucial step in the development of PHP applications that use a MySQL database. Through user research, prototyping, and A/B testing, developers can identify any shortcomings, usability issues, and design inconsistencies [7]. These tests allow developers to gather valuable feedback from real users and identify areas that require improvement or additional iterations in the design and functionality of the application. Through continuous testing, the application can be improved to ensure the best possible user experience and user satisfaction [8].

Continuous improvement of the user interface is a key practice that allows the application to remain relevant and competitive in a dynamic digital environment. By analyzing usage data, user feedback, and changes in technology, developers can identify new opportunities to improve the user

experience and implement them into the application. An iterative approach to development allows the application to evolve and adapt to the needs of users, resulting in long-term user satisfaction and loyalty to the application.

3. TOOLS FOR USER INTERFACE DEVELOPMENT

Tools for user interface development play a crucial role in the process of creating PHP applications that utilize a MySQL database [9]. There is a wide range of tools and technologies available to developers that facilitate the development and design of user interfaces, providing them with the necessary resources and functionalities to create modern and functional web applications [10]. These tools include programming languages such as HTML, CSS, and JavaScript, which allow for defining the structure, styling, and interactivity of the user interface [11]. Additionally, frameworks and libraries like Bootstrap, jQuery, and React provide ready-made components, patterns, and styles that facilitate development, simultaneously improving productivity and code quality [12]. Moreover, interface design tools such as Adobe XD, Sketch, and Figma enable designers to prototype, design, and test the user interface before implementation begins [13]. The combination of these tools allows developers to create PHP applications with a MySQL database that are not only functional and efficient but also aesthetically appealing and user-oriented.

3.1. An overview of PHP Environment

Overview of the PHP Environment is a crucial step in the development of PHP applications that utilize a MySQL database. PHP is a server-side scripting language commonly used for dynamically generating web content. The PHP environment encompasses several components that enable the execution of PHP code on a web server. The core components include a web server, PHP interpreter, and MySQL database [14]. A web server, such as Apache, Nginx, or IIS, serves to receive requests from clients and execute PHP scripts that generate dynamic HTML content [15]. The PHP interpreter processes PHP code and generates HTML sent to the user via the web server. The MySQL database enables storage, management, and manipulation of data used in PHP applications [16]. In addition to these core components, the PHP environment may include additional tools, libraries, and frameworks that facilitate the development and maintenance of PHP applications, such as Composer for dependency management, PHPUnit for testing, and Laravel or Symfony for PHP framework development [17]. An overview of the PHP environment enables programmers to understand the architecture and components necessary for developing PHP applications that efficiently utilize a MySQL database, resulting in stable, scalable, and secure web applications.

3.2. Database Management System - MySQL

MySQL is one of the most popular RDBMS (Relational Database Management Systems) used in web application development [18]. As an open-source system, MySQL is available on various platforms and provides a high level of flexibility, enabling efficient data management in PHP applications. MySQL uses SQL (Structured Query Language) for data manipulation, offering support for transactions, indexing, security mechanisms, and data replication.

MySQL allows users to efficiently store and manage data organized into tables. Databases are crucial for processing large amounts of information, making them essential for many applications, including PHP applications. MySQL enables users to easily manage database access, reducing the risk of errors, and is most commonly used in combination with PHP on Apache web servers [19]. On a server, multiple databases can exist independently, and different administrative rights can be assigned to each user. MySQL creates a super administrator (usually root) with all rights, including the ability to create, modify, and delete databases and tables. MySQL is available for all major operating systems and is distributed under the GPL (General Public License), making it ideal for learning and developing smaller to medium-sized websites.

The MySQL installation includes several key folders: bin (with server and client programs), lib (function libraries), scripts (Perl scripts), share (textual error message files), and include (header files for compilation). Executable files are located in bin and scripts folders, including mysqld (server), mysqladmin (administrative functions), myisamchk (table check and repair), mysqldump (backup), and mysqlshow (displaying database and table data).

4. DEVELOPMENT OF THE USER INTERFACE

This section explores a comprehensive approach to designing the user interface for PHP applications that use MySQL databases. The focus is on creating an intuitive and pleasant user experience that meets the users' needs. Through analyzing user requirements, identifying key functionalities, and creating prototypes, developers can develop a structure, layout, and interactivity of the interface that fulfills the project's goals.

By utilizing modern technologies and tools such as HTML, CSS, JavaScript, jQuery, and others, the development team can create a dynamic and attractive user interface [20]. This interface enables users to easily manage data and interact with the application. Additionally, part of the research focuses on strategies for testing the user interface, identifying shortcomings, and iteratively improving it to achieve optimal user experience.

The focus on developing a user-centric, aesthetically appealing, and functional user interface allows PHP applications that use MySQL databases to achieve a high level of success and user satisfaction.

5. IMPLEMENTATION OF AN APPLICATION FOR WORKING WITH A DATABASE

The idea for creating a PHP application stemmed from the need to customize the interface within the PHPMyAdmin environment for working with MySQL databases on the server. When PHPMyAdmin is launched, in addition to user databases, system databases necessary for MySQL and PHP server operation also appear. To prevent errors and damage to system databases, a PHP application specialized for working with user databases was developed, as shown in Figure 1.

This application allows for the creation, deletion, and addition of tables in databases, as well as modifications to the database structure. Administrative access to system databases is disabled because the application does not load those databases upon startup, thereby increasing security and reducing the risk of errors.

WORKING WITH DATABASES IN A PHP ENVIRONMENT

The screenshot displays a web-based interface for managing MySQL databases. It is divided into several sections:

- DATABASES ON THE SERVER:** A list of databases including 'information_schema', 'mysql', 'performance_schema', 'phpmyadmin', 'test', and 'user'. A 'Create new database' button is present.
- SELECTED DATABASE TABLES:** A list of tables for the selected database, including 'columns_privileges', 'columns_statistics', 'columns_statistics_history', 'columns_statistics_history_index', and 'columns_statistics_history_index_index'. A 'Create new table' button is present.
- STRUCTURE OF THE SELECTED TABLE:** A table structure editor showing columns, types, and lengths. It includes buttons for 'Changing the field structure', 'Add field', and 'Clear field'.
- Creating a new database table:** A form to enter the name of the table to be created.
- Deleting a database table:** A form to enter the name of the table to be deleted.

Figure 1. Database Management Application

The application design is based on displaying databases on the server, a command block for creating and deleting databases, a list of tables for the selected database, a command button for creating a new table in the selected database, the structure of the selected table, and a command block for editing the structure of the selected database table, as shown in Figure 1. It is important to note that data manipulation commands for the database table are not included within the application; in other words, the purpose of this application is to facilitate the work of database administrators. The application is launched by entering a specific address into a web browser:

<http://localhost:8082/tie/indexcb.php>

Due to the complexity of the application, in addition to the main program indexcb.php, four additional subprograms have been developed, PHP-1, which are called from the main program. These subprograms include:

- ConnectToServer.php
- DatabaseArray.php

- ChangeTable.php
- Add_Lists.php

PHP - 1

```
1 <?php
2 include("ConnectToServer.php");
3 include("DatabaseArray.php");
4 include("ChangeTable.php");
5 include("Add_Lists.php");
6 ...
7 ?>
```

By using the include command (lines 2 to 5), the PHP compiler is informed that the functions called in the main program are located in one of the specified PHP files.

It is important to note that when performing actions via command buttons, the current page is not left; instead, it is refreshed after each executed command. To achieve this, it is necessary to explicitly specify which action should be taken when an event occurs on the page in the form definition. The command from the PHP server `$_SERVER['PHP_SELF']` is executed as shown. The method used for transmitting executed actions is POST.

5.1. Establishing connection to the database server

For the successful operation of this application, as well as any other application working with databases, it is necessary to establish a connection to the database server, as shown in PHP code - 2. In this example, the database server is MySQL, as already noted. To establish a connection to the database server, the function `ConnectToServer()` is created, which is located in the program `ConnectToServer.php`. This function is called from the main program or subprograms whenever it is necessary to establish a connection to any of the databases on the server, i.e., before executing any action on the database or its tables. To open the desired database, in addition to the server, it is necessary to specify the name of the database.

PHP - 2

```
1 <?php
2 function ConnectToServer ()
3 {
4     $connection=mysqli_connect("localhost",
5     "root","");
6     if($connection)
7     {
8         return $ connection;
9     }
10    else
11    {
12        exit("Failed to connect to the MySQL server
13        ");
```

```
12 }
13 }
14 ----
15 ?>
```

5.2. Creating a list of databases on the server

To form a list of databases located on the server, after establishing a connection to the server, it is necessary to execute a query that will load all user databases but exclude system databases. The code achieving this is provided in PHP - 3.

PHP - 3

```
1 <?php
2 include("ConnectToServer.php");
3 include("DatabaseArray.php");
4 include("PromenaTabele.php");
5 include("ChangeTable.php");
6 ---
7 $ Array_Type = array("varchar", "text",
8 "char","int","float","decimal","date",
9 "datetime");
10 $ Array_021[]= array("id" => 0, "rbt" =>
11 0, "tabela" => "");
12 $ Array_02[] = array("id" =>0, "val" =>
13 "");
14 $ Array_03[] = array("rbt" => "", "field" =>
15 "", "tip" => "");
16 $ Array_01[] = array("id" => 0, "val" =>
17 "", "tableCount "=>0);
18 for($i=0;$i<100;$i++)
19 {
20     $ StructureArray[$i]="---";
21 }
22 ---
23 Nizovi($Array_01, $Array_021, $ Array_02,
24 $ Array_03, $CountBP);
25 ---
26 ?>
```

The query results are stored in the corresponding arrays. Initialized arrays (lines 7 to 14) hold the following data:

- `$Array_01[]` - user databases on the server,
- `$Array_02[]` - tables within the selected database,
- `$Array_03[]` - structure of the selected database table.

The Arrays function is called with appropriate parameters, which are updated within the function itself. The implementation of this function is located in the `Arrays.php` program.

Connecting to the database server is performed while defining the query string that will load all databases from the server. The query result is stored in the `$result` variable. This variable contains names of both user and system databases, so it is

necessary to filter the result to load only user databases into the auxiliary array. After that, the previously established connection is released to prepare a new query for reading tables for each of the databases stored in the array.

In this way, the arrays \$Array_01[] - user databases on the server, \$Array_02[] - tables within the selected database, and \$Array_03[] - structure of the selected database table are populated.

5.3. Filling lists of databases, tables, and table structures

After the appropriate arrays have been populated, it is necessary to display the data about databases, tables for the selected database, and the structure of the selected table.

An object ListaBaza is defined with the procedure this.form.submit(), which checks if the user has selected a row, i.e., a database. Forming rows in the list is done by calling the PHP function add_database_list() with the appropriate parameters: \$selectedDB - a variable containing the name of the selected database from the list, and the name of the array \$Array_01 from which the names of user databases are read. The \$selectedDB variable enables selecting the previously chosen database when refreshing the page.

5.4. Creating/Deleting a Database

Creating a new database starts with entering the database name and clicking on the button to create the database. If the database is successfully created, an appropriate message will appear, and after refreshing the page, the name of the new database will appear in the list of databases.

To delete a database from the server, you need to select the database you want to delete from the list of databases and click on the delete button. After performing this action, the selected database will be deleted from the server, and its name will disappear from the list of databases after refreshing the page. On Figure 2, the creation of a database in the application is illustrated.

WORKING WITH DATABASES IN A PHP ENVIRONMENT

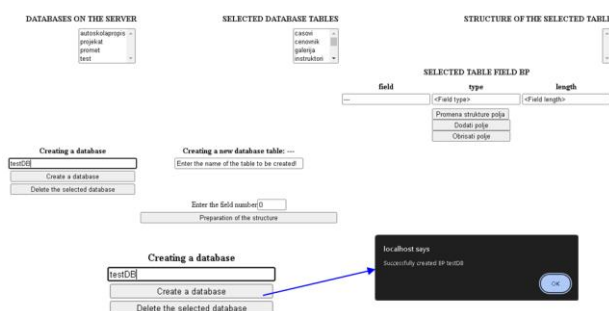


Figure 2. Creating a Database

5.5. Creating and Modifying a Database Table

Selecting a database from the database list displays a list of tables in the selected database, and selecting a specific table populates the list with its structure. Choosing a field from the list populates the columns with the field name, type, and length.

To modify the table structure, you can change the type and length of a field and then click the "Change field structure" button, or you can enter a new name, type, and length for a field and then click "Add field". To delete a field, click "Delete field". To add a new table, start by entering the table name and defining the number of fields, then click the "Prepare structure" button. After this action, rows for entering the field name, type, and length are generated. Clicking the "Create table" button creates a table with the entered structure, which will appear in the list after refreshing, as shown in Image 3.



Figure 3. Creating a database table and selecting the field type

5.6. Discussion

This paper presents the process of creating a PHP user interface for manipulating MySQL databases, focusing on basic functionalities such as reading, writing, updating, and deleting data. To provide a broader context and highlight the advantages of the developed application, it is useful to compare it with existing user interfaces like phpMyAdmin, Adminer, and MySQL Workbench. phpMyAdmin is known for its ease of use and wide range of functionalities, allowing users to easily execute SQL queries, import and export data, and manage users. However, it can be slow when handling large databases and is often targeted by attacks, requiring careful configuration and regular updates. Adminer is easier and quicker to set up, with lower server resource requirements, but lacks some advanced features offered by phpMyAdmin, limiting its use for more complex tasks. MySQL Workbench offers advanced functionalities and better performance for working with large databases, but as a desktop application, it is not as accessible as web-based tools like phpMyAdmin and Adminer.

The created PHP user interface for manipulating MySQL databases offers several unique advantages. Firstly, it provides flexibility by allowing the creation of a custom interface that can be precisely tailored to the specific needs of users and applications. Additionally, it focuses on security, enabling design with specific security measures that reduce risks associated with general-purpose tools. Moreover, the interface is optimized for performance, particularly for key queries and operations that are significant for users.

Like any customized tool, the user interface also has its limitations. Development and maintenance require more time and resources compared to using existing tools. Furthermore, creating advanced functionalities can be complex and require a high level of technical knowledge to meet specific user and application requirements.

6. CONCLUSION

This paper explores key aspects of creating a PHP user interface for working with MySQL databases. Through an analysis of interface design, development tools, testing, and enhancement, as well as a review of the PHP environment and MySQL database management system, we uncover the importance of integrating these two technological pillars in web application development. Implementing an efficient user interface enables users to intuitively and effectively manage data, while PHP and MySQL provide a robust foundation for secure and scalable operation. Through proper design, development, and testing of the application, it is possible to enhance the user experience and optimize system performance. Overall, this paper contributes to understanding the complexity and significance of creating a PHP user interface for working with MySQL databases, highlighting the importance of this technological combination in modern web development.

This paper presents the process of creating a database management application in a PHP environment. Starting from the defined graphical interface for database administration within the phpMyAdmin software, the basic idea was to create an environment that would allow database administrators to work only with user databases. This is important considering the possibility of modifying the mentioned software and system databases necessary for MySQL and PHP operation. Such a customized environment facilitates database administration, increasing efficiency and reducing the risk of unwanted changes.

REFERENCES

- [1] Nixon, R. (2014). *Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5*. "O'Reilly Media, Inc."
- [2] Milošević, D., Pepić, S., Saračević, M., & Tasić, M. (2016). *Weighted Moore-Penrose generalized matrix inverse: MySQL vs. Cassandra database storage system*. *Sādhanā*, 41, 837-846.
- [3] Meloni, J. C. (2008). *Sams teach yourself PHP, MySQL and Apache all in one*. Pearson Education India.
- [4] Mojsilović, M., Pepić, S., & Miodragović, G. (2022). *Implementation of embedded messages using steganography in the PHP software package*. *Technics and Informatics in Education-TIE*, 359-370.
- [5] Moore, R. J., Arar, R., Ren, G. J., & Szymanski, M. H. (2017, May). *Conversational UX design*. In *Proceedings of the 2017 CHI conference extended abstracts on human factors in computing systems* (pp. 492-497).
- [6] Wu, D. (2022). *The application and management system of scientific research projects based on PHP and MySQL*. *Journal of Interconnection Networks*, 22(Supp02), 2143043.
- [7] Miller, D. (2021). *The best practice of teach computer science students to use paper prototyping*. *International Journal of Technology, Innovation and Management (IJTIM)*, 1(2), 42-63.
- [8] Mochammad Aldi Kushendriawan, M. A. K., Harry Budi Santoso, H. B. S., Putra, P. O. H., Putra, P. O. H., & Martin Schrepp, M. S. (2021). *Evaluating User Experience of a Mobile Health Application Halodoc using User Experience Questionnaire and Usability Testing*. *Jurnal Sistem Informasi (Journal of Information System)*, 17(1), 58-71.
- [9] Amini, M., Rahmani, A., Abedi, M., Hosseini, M., Amini, M., & Amini, M. (2021). *Mahamgostar. Com as a case study for adoption of laravel framework as the best programming tools for php based web development for small and medium enterprises*. *Journal of Innovation & Knowledge*, ISSN, 100-110.
- [10] Tatroe, K., & MacIntyre, P. (2020). *Programming PHP: Creating dynamic web pages*. O'Reilly Media.
- [11] Tasić, M. B., Stanimirović, P. S., & Pepić, S. H. (2011). *Computation of generalized inverses using Php/MySql environment*. *International Journal of Computer Mathematics*, 88(11), 2429-2446.
- [12] Verma, A., Pandey, B., Gupta, C., & Tanwar, V. K. (2022). *Installation of Latest Version of NodeJS, NPM, Angular, Bootstrap, jQuery, Typescript, React in Windows 7*. *NeuroQuantology*, 20(7), 2305.
- [13] Wang, J., Xu, Z., Wang, X., & Lu, J. (2022). *A comparative research on usability and user experience of user interface design software*. *International Journal of Advanced Computer Science and Applications*, 13(8).
- [14] Williams, H. E., & Lane, D. (2004). *Web Database Applications with PHP and MySQL: Building Effective Database-Driven Web Sites*. "O'Reilly Media, Inc."

- [15]Agustine, L., & Seimahuira, S. (2023). *Penerapan Metode SAW dalam Analisa Perbandingan Performa Web server (Apache, Nginx, Lighttpd, Iis) pada Bahasa Pemrograman PHP*. REMIK: Riset dan E-Jurnal Manajemen Informatika Komputer, 7(1), 409-420.
- [16]Milivojević, M., & Zeljković, Ž. (2020). *Izrada prototipa u softverskom alatu adobe experience design*. Zbornik radova Fakulteta tehničkih nauka u Novom Sadu, 35(02), 250-253.
- [17]Shestakov, Y. (2020). *Development of robust SDKs for REST APIs in PHP*. How to effectively develop, maintain and release REST API SDKs.
- [18]Khan, M. Z., Zaman, F. U., Adnan, M., Imroz, A., Rauf, M. A., & Phul, Z. (2023). *Comparative Case Study: An Evaluation of Performance Computation Between SQL And NoSQL Database*. Journal of Software Engineering, 1(2), 14-23.
- [19]Islam, K., Ahsan, K., Bari, S. A. K., Saeed, M., & Ali, S. A. (2017). *Huge and Real-Time Database Systems: A Comparative Study and Review for SQL Server 2016, Oracle 12c & MySQL 5.7 for Personal Computer*. Journal of Basic & Applied Sciences, 13(21), 481-490.
- [20]Eisenberg, A., & Melton, J. (2000). *SQL standardization: the next steps*. ACM SIGMOD Record, 29(1), 63-67.